

| React 물리엔진 구현

| 📝 Log: <% tp.file.title %>

| 📁 상위 활동: [NP1](#)

| 📁 Assignments

☐ 과제 1:

☐ 과제 2:

| ✅ To-Do List

☐

☐

| 📄 Content & Progress

| React에서 Matter.js와 useState를 활용한 물리엔진 구현 가이드

| 1. 개요

이 가이드는 물리엔진인 `Matter.js` 를 React 환경에 통합하고, React의 상태 관리 혹은 `useState` 를 이용해 UI와 물리엔진 세계(World)를 상호작용하게 만드는 방법을 다룹니다.

- **Matter.js의 특징:** DOM을 직접 조작하여 Canvas에 물리 엔진을 렌더링합니다.
- **React의 특징:** 가상 DOM(Virtual DOM)을 통해 상태(`state`) 기반으로 UI를 렌더링합니다.
- **핵심 전략:** 두 환경이 충돌하지 않도록 Matter.js의 Canvas는 `useRef` 로 독립적인 공간에 렌더링하고, 버튼 클릭 같은 React의 `useState` 변경 사항을 `useEffect` 나 이벤트 핸들러를 통해 Matter.js World에 주입합니다.

| 2. 프로젝트 세팅

먼저 React 프로젝트를 생성하고 `matter-js` 라이브러리를 설치해야 합니다.

Bash

```
# Vite를 이용한 React 프로젝트 생성 (권장)
npm create vite@latest matter-react-app --template react
cd matter-react-app

# matter-js 설치
npm install matter-js
```

3. 핵심 구현 코드 (App.jsx)

아래 코드는 하늘에서 상자가 떨어지는 물리 엔진을 구현하고, React의 `useState` 를 통해 화면 상단의 '상자 개수' 상태를 관리하며, 버튼을 눌러 상자를 동적으로 추가하는 예제입니다.

JavaScript

```
import React, { useEffect, useRef, useState } from 'react';
import Matter from 'matter-js';

function App() {
  // React State: UI에 표시될 상자의 개수 관리
  const [boxCount, setBoxCount] = useState(0);

  // DOM 참조 및 Matter.js 인스턴스 보관용 Ref
  const sceneRef = useRef(null);
  const engineRef = useRef(null);

  useEffect(() => {
    // 1. Engine 및 World 초기화
    const Engine = Matter.Engine;
    const Render = Matter.Render;
    const Runner = Matter.Runner;
    const Bodies = Matter.Bodies;
    const Composite = Matter.Composite;

    const engine = Engine.create();
    engineRef.current = engine; // 다른 함수에서 엔진에 접근하기 위해 저장

    // 2. Render 셋업 (useRef로 연결된 div에 Canvas 생성)
    const render = Render.create({
      element: sceneRef.current,
      engine: engine,
      options: {
        width: 800,
        height: 600,
        wireframes: false, // true로 하면 색상 없이 뼈대만 보임
        background: '#f8f9fa'
      }
    });

    // 3. 정적 바닥(Ground) 생성 (x, y, width, height, options)
    const ground = Bodies.rectangle(400, 590, 810, 60, { isStatic: true,
      render: { fillStyle: '#343a40' } });
    Composite.add(engine.world, [ground]);

    // 4. 실행
    Render.run(render);
    const runner = Runner.create();
```

```

Runner.run(runner, engine);

// 5. Cleanup 함수 (React 컴포넌트 언마운트 시 메모리 누수 방지)
return () => {
  Render.stop(render);
  Runner.stop(runner);
  Composite.clear(engine.world);
  Engine.clear(engine);
  render.canvas.remove();
  render.canvas = null;
  render.context = null;
  render.textures = {};
};
}, []); // 빈 배열을 넣어 마운트 시 1회만 실행되도록 함

// 버튼 클릭 시 Matter.js 세계에 상자를 추가하고 React 상태를 업데이트하는 함수
const handleAddBox = () => {
  if (!engineRef.current) return;

  // 랜덤한 X 위치에서 떨어지도록 설정
  const randomX = Math.random() * 600 + 100;

  // Matter.js 물체 생성
  const newBox = Matter.Bodies.rectangle(randomX, 50, 40, 40, {
    restitution: 0.8, // 탄성 (통통 튀는 정도)
    render: { fillStyle: '#4dabf7' }
  });

  // 물리 엔진 World에 추가
  Matter.Composite.add(engineRef.current.world, newBox);

  // React State 업데이트
  setBoxCount((prevCount) => prevCount + 1);
};

return (
  <div style={{ display: 'flex', flexDirection: 'column', alignItems:
'center', padding: '20px' }}>
    <h1>React + Matter.js 연동</h1>

    { /* React의 상태(useState)가 반영되는 UI 구역 */ }
    <div style={{ marginBottom: '20px' }}>
      <p>현재 떨어뜨린 상자 개수: <strong>{boxCount}</strong>개</p>
      <button
        onClick={handleAddBox}
        style={{ padding: '10px 20px', cursor: 'pointer', fontSize:
'16px' }}
      >
        상자 떨어뜨리기!
      </button>

```

```

    </div>

    { /* Matter.js가 그려질 Canvas 컨테이너 */ }
    <div
      ref={sceneRef}
      style={{ border: '2px solid #ccc', borderRadius: '8px', overflow:
'hidden' }}
    ></div>
  </div>
);
}

export default App;

```

4. 구현 포인트 해설

- 렌더링 분리:** React는 `div` 컨테이너(`sceneRef`)만 제공하고, 내부 Canvas 렌더링은 전적으로 Matter.js에게 위임합니다. 이렇게 해야 React가 상태를 업데이트할 때마다 Canvas 전체를 리렌더링하여 물리 엔진이 끊기는 현상을 막을 수 있습니다.
- useRef 의 활용:** `engineRef` 를 통해 `useEffect` 내부에서 생성된 `engine` 객체를 컴포넌트 생명주기 동안 유지합니다. 이를 통해 `handleAddBox` 함수 내부에서 `engine.world` 에 접근하여 새로운 객체를 동적으로 추가할 수 있습니다.
- useState 와의 통신:** 사용자가 버튼을 클릭하면, 1) Matter.js 세계에 물리 객체가 그려지고, 2) React의 `boxCount` State가 변경되어 UI에 상자 개수가 텍스트로 업데이트됩니다. 즉, 물리 세계의 로직과 UI 로직이 비동기적으로 조화롭게 동작합니다.

```
<%* await tp.file.move("20.Study & Career/60. Acting Log/" + tp.file.title); %>
```